Πολυτεχνείο Κρήτης
Σχολή ΗΜΜΥ
Στατιστική Μοντελοποίηση και Αναγνώριση Προτύπων
Παράδοση 2ης εργασίας
Ημερομηνία Παράδοσης: 12 Ιουλίου 2023

Ομάδα 33

	Φοιτητής 1	Φοιτητής 2
Επώνυμο	Αντωνογιαννάκης	Χαϊντούτη
Όνομα	Ιωάννης	Μαρία
Α.Μ.	2020030018	2020030129

Θέμα 1:Λογιστική Παλινδρόμηση: Αναλυτική εύρεση κλίσης (Gradient).

α. Αρχικά, υπολογίζεται η κλίση του σφάλματος $J(\theta)$:

$\nabla_\theta(J(\theta)) = \nabla_\theta(\frac{1}{m}\sum_{i=1}^{m} -y^{(i)} \cdot \ln(h_\theta(x^{(i)})) - (1-y^{(i)}) \cdot \ln(1-h_\theta(x^{(i)}))$
Λόγω της γραμμικότητας του gradient και του $\sum$ έχουμε ότι:

$\nabla_\theta(J(\theta)) = \frac{1}{m}\sum_{i=1}^{m} \nabla_\theta(-y^{(i)} \cdot \ln(h_\theta(x^{(i)})) - (1-y^{(i)}) \cdot \ln(1-h_\theta(x^{(i)}))) =$
$= \frac{1}{m}\sum_{i=1}^{m}(-y^{(i)} \cdot \nabla_\theta \ln(h_\theta(x^{(i)})) - (1-y^{(i)}) \cdot \nabla_\theta \ln(1-h_\theta(x^{(i)})))$

Τα δύο Gradients υπολογίζονται ξεχωριστά:

- $\ln(h_\theta(x^{(i)})) = \ln(\frac{1}{1+e^{-\theta^T x}}) = \ln 1 - \ln(1+e^{-\theta^T x})$
  Άρα: $\nabla_\theta(\ln(h_\theta(x^{(i)}))) = \nabla_\theta(\ln(1+e^{-\theta^T x}))$
  $= \frac{-1}{1+e^{-\theta^T x}} \frac{\partial(1+e^{-\theta^T x})}{\partial(-\theta^T)} = \frac{-1}{1+e^{-\theta^T x}} e^{-\theta^T x} \nabla_\theta(-\theta^T)$
  $= \frac{-1}{1+e^{-\theta^T x}} e^{-\theta^T x}(-x) = \frac{xe^{-\theta^T x}}{1+e^{-\theta^T x}}$
- $\nabla_\theta(1-h_\theta(x^{(i)})) = \frac{1}{1-h_\theta(x^{(i)})} = \frac{1}{1-\frac{1}{1+e^{-\theta^T x}}} \cdot \nabla_\theta(1-\frac{1}{1+e^{-\theta^T x}}) =$
  $\frac{1+e^{-\theta^T x}}{e^{-\theta^T x}} \cdot \nabla_\theta(\frac{e^{-\theta^T x}}{1+e^{-\theta^T x}} = \frac{1+e^{-\theta^T x}}{e^{-\theta^T x}} \cdot \frac{e^{-\theta^T x}+e^{-2\theta^T x}-e^{-2\theta^T x}}{(1+e^{-\theta^T x})^2} \cdot \nabla_\theta(-\theta^T x)$
  $= \frac{1+e^{-\theta^T x}}{e^{-\theta^T x}} \cdot \frac{e^{-\theta^T x}}{(1+e^{-\theta^T x})^2} \cdot (-x) = \frac{-x}{1+e^{-\theta^T x}}$

Άρα, λοιπόν:
$\nabla_\theta(J(\theta)) = \frac{1}{m}\sum_{i=1}^{m} \frac{-y^{(i)}x^{(i)}e^{-\theta^T x}}{1+e^{-\theta^T x}} - (1-y^{(i)})\frac{-x^{(i)}}{1+e^{-\theta^T x}} =$
$= \frac{1}{m}\sum_{i=1}^{m} -y^{(i)}x^{(i)}e^{-\theta^T x}h_\theta(x^{(i)} + x^{(i)}(1-y^{(i)})h_\theta(x^{(i)}) =$
$= \frac{1}{m}\sum_{i=1}^{m} x^{(i)}[-y^{(i)}e^{-\theta^T x}h_\theta(x^{(i)} + h_\theta(x^(i)) - y^{(i)}h_\theta(x^{(i)})] =$
$= \frac{1}{m}\sum_{i=1}^{m} x^{(i)}[h_\theta(x^(i)) - y^{(i)}(1+e^{-\theta^T x})h_\theta(x^{(i)})$,
όπου $1+e^{-\theta^T x^{(i)}} = \frac{1}{h_\theta(x^{(i)})}$.

Επομένως: $\nabla_\theta(J(\theta)) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x^{(i)}$

Το j-στοιχείο του gradient, συνεπώς, είναι:

$\frac{\partial(J(\theta))}{\partial(\theta_j)} = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x_j^{(i)}) - y^{(i)})x_j^{(i)}$

β. Στο β μέρος θα χρησιμοποιήσουμε την λογιστική παλινδρόμηση για να προβλέψουμε αν ένας φοιτητής θα γίνει δεκτός σε ένα πανεπιστήμιο με βάση τους βαθμούς του σε δύο εξετάσεις.

i) Αρχικά επεικονίσαμε τα δεδομένα στο δισδιάστατο επίπεδο.

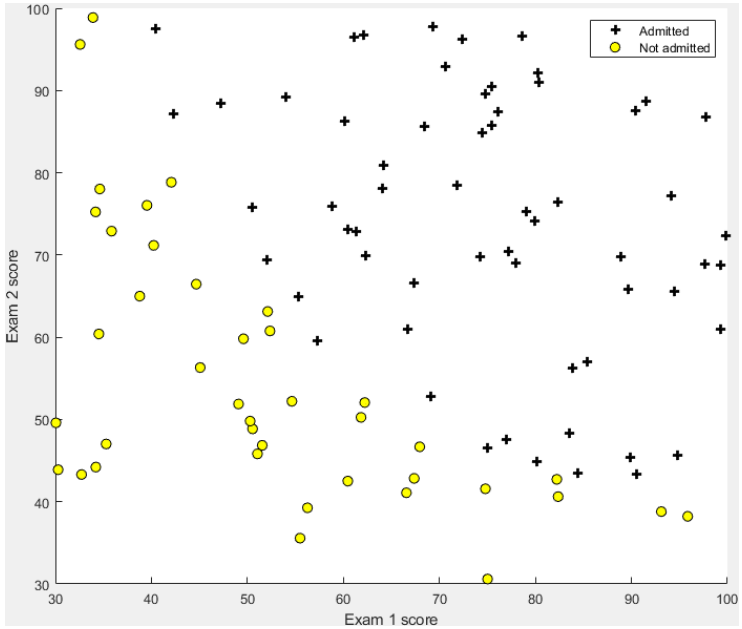


Figure 1: Δεδομένα της άσκησης.

ii) Υλοποιήστε την σιγμοειδή συνάρτηση $f(z)$ χρησιμοποιώντας τον τύπο

$$f(z) = \frac{1}{1+e^{-z}}$$

iii) Υλοποιήσαμε την συνάρτηση costFunction, ώστε να επιστρέφει το κόστος $J(\theta)$ και την κλίση του, σύμφωνα με τον τύπο του cross-entropy.
για $\theta = 0$ επιβεβαιώσαμε τις τιμές $J = 0.693$ και $\nabla J =$ [-0.1, -12.009217, -11.262842]

```
Cost at initial theta (zeros): 0.693147
Gradient at initial theta (zeros):
 -0.100000
 -12.009217
 -11.262842
```

iv) Τέλος τρέξαμε τον κώδικα και βρήκαμε το σύνορο απόφασης. Επίσης συμπληρώσαμε κώδικα στο αρχείο predict.m, ώστε να προβλέψουμε αν ο φοιτητής θα γίνει δεκτός με βάση διάφορες τιμές βαθμών στις δύο εξετάσεις.

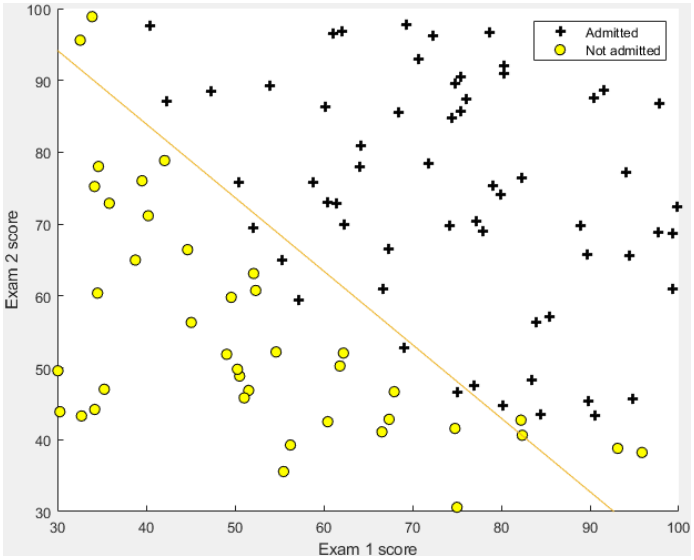


Figure 2: Δεδομένα και όριο απόφασης.

- Για έναν μαθητή με βαθμούς 45 και 85 υπολογίζουμε πιθανότητα εισαγωγής 0.776291.
- Η ακρίβεια για τα δεδομένα εκπαίδευσης είναι 89.000000%.

Θέμα 2:Λογιστική Παλινδρόμηση με Ομαλοποίηση.

a. Αρχικά απεικονίστηκαν τα δεδομένα σε 2-D επίπεδο:

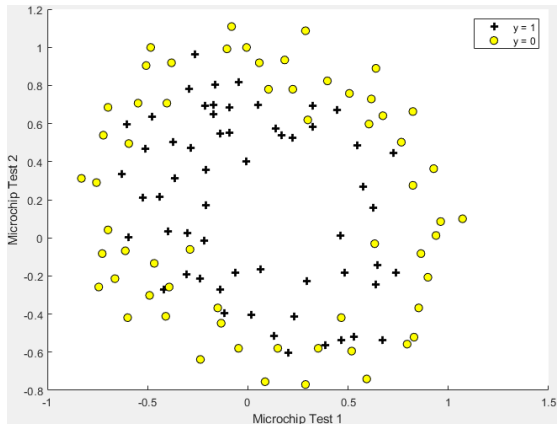


Figure 3: Δεδομένα των δοκιμών σε μικροτσίπ.

Το κόστος για το αρχικό ϑ είναι ίσο με: $J_{init} = 0.693147$

b. Συμπληρώθηκε κώδικας στη συνάρτηση mapFeature και την χρησιμοποιήσαμε, ώστε να προβάλλουμε τα δεδομένα σε χώρο μεγαλύτερων διαστάσεων.

c. Η ομαλοποιημένη συνάρτηση κόστους δίνεται από την σχέση:
$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}(-y^{(i)}\cdot\ln(h_\theta(x^{(i)}))-(1-y^{(i)})\cdot\ln(1-h_\theta(x^{(i)}))+\frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$

d. Υπολογίζεται για αρχή, η κλίση του σφάλματος $J(\theta)$:
$\nabla_\theta(J(\theta)) =$
$\nabla_\theta(\frac{1}{m}\sum_{i=1}^{m} -y^{(i)}\cdot\ln(h_\theta(x^{(i)})) - (1-y^{(i)})\cdot\ln(1-h_\theta(x^{(i)})) + \frac{\lambda}{2m}\sum_{j=1}^{n}\theta^2)$
$= \frac{1}{m}\sum_{i=1}^{m}(-y^{(i)}\nabla_\theta\ln(h_\theta(x^{(i)})) - (1-y^{(i)})\nabla_\theta\ln(1-h_\theta(x^{(i)})) +$
$\frac{\lambda}{2m}\sum_{j=1}^{n}\nabla_\theta\theta^2)$
Υπολογίζονται ξεχωριστά τα gradients:

- $\ln(h_\theta(x^{(i)})) = \ln(\frac{1}{1+e^{-\theta^T x}}) = \ln 1 - \ln(1+e^{-\theta^T x})$

  Άρα, όπως στο ϑέμα 1: $\nabla_\theta(\ln(h_\theta(x^{(i)}))) = \frac{xe^{-\theta^T x}}{1+e^{-\theta^T x}}$
- $\nabla_\theta(1-h_\theta(x^{(i)})) = \frac{-x}{1+e^{-\theta^T x}}$, όπως και στο ϑέμα 1.
- $\nabla_\theta(\theta^2) = 2\theta$

Τελικά, η κλίση του σφάλματος είναι:

$\nabla_\theta J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \frac{-y^{(i)}x^{(i)}e^{-\theta^T x}}{1+e^{-\theta^T x}} - (1-y^{(i)})\frac{-x^{(i)}}{1+e^{-\theta^T x}} + \frac{\lambda}{2m}2\theta$

$= \frac{1}{m}\sum_{i=1}^{m} x^{(i)}[h_\theta(x^{(i)}) - y^{(i)}(1+e^{-\theta^T x})h_\theta(x^{(i)}) + \frac{\lambda}{m}\theta$

$\Leftrightarrow \nabla_\theta(J(\theta)) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x^{(i)} + \frac{\lambda}{m}\theta$

Επομένως, το στοιχείο j του gradient είναι:

$\frac{\partial(J(\theta))}{\partial(\theta_j)} = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x_j^{(i)}) - y^{(i)})x_j^{(i)} + \frac{\lambda}{m}\theta_j$

e. Συμπληρώσαμε τον κώδικα στη συνάρτηση costRegFunction, ώστε αυτή να υπολογίζει το κόστος, καθώς και την κλίση του (gradient).

f.
- για λ=0:

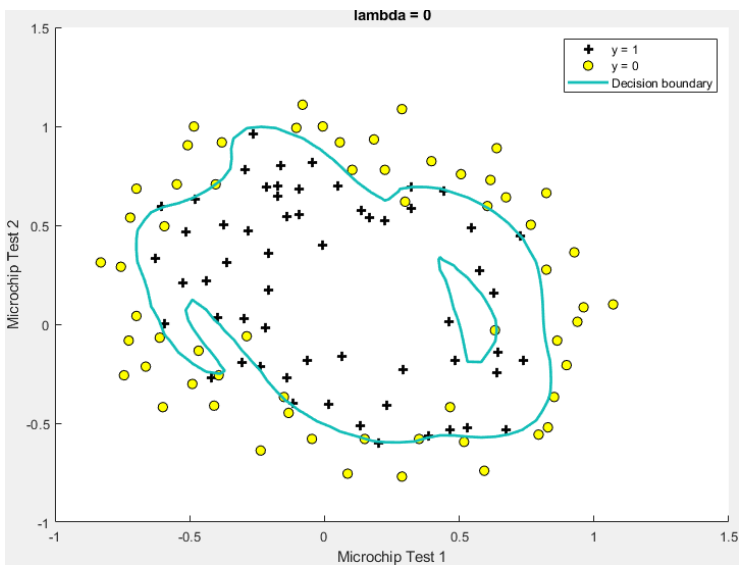


Figure 4: Σύνορο απόφασης για λ=0.

Η ακρίβεια πρόβλεψης των training δεδομένων μετρήθηκε στο: 88.98%.

- για λ=1:

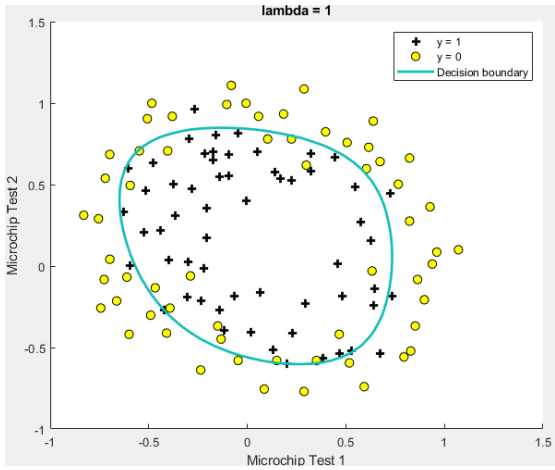


Figure 5: Σύνορο απόφασης για λ=1.

Η ακρίβεια πρόβλεψης των training δεδομένων μετρήθηκε στο: 82.02%.

- για λ=10:

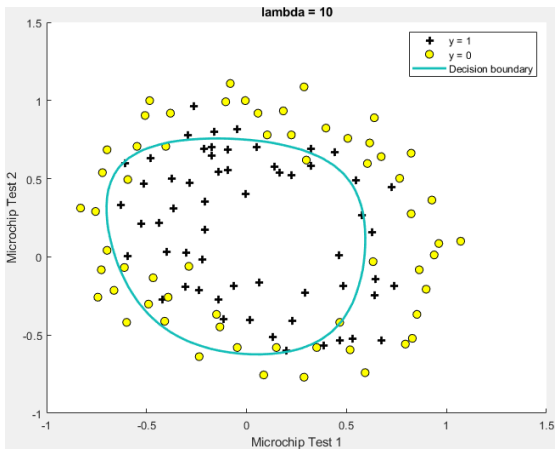


Figure 6: Σύνορο απόφασης για λ=10.

Η ακρίβεια πρόβλεψης των training δεδομένων μετρήθηκε στο: 74.58%.

- για λ=100:

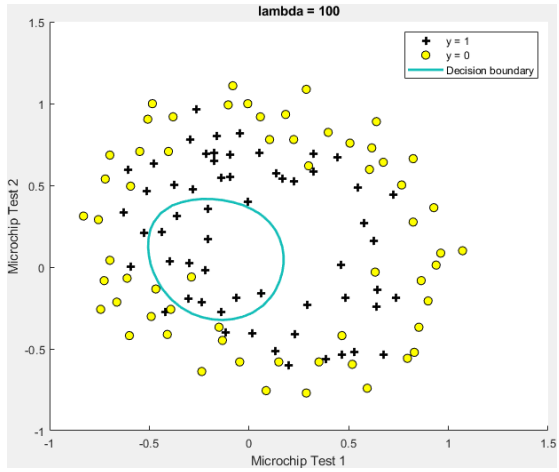


Figure 7: Σύνορο απόφασης για λ=100.

Η ακρίβεια πρόβλεψης των training δεδομένων μετρήθηκε στο: 60.17%.

Παρατηρούμε ότι όσο αυξάνεται η τιμή του λ τόσο χειρότερο γίνεται το όριο απόφασης με συνέπεια να μειώνεται και η ακρίβεια.

Θέμα 3: Εκτίμηση Παραμέτρων με Maximum Likelihood και MAP.

A. Ο κανόνας μέγιστης πιθανοφάνειας (MLE) είναι ο εξής: $\hat{\lambda_{ML}} = argmax_{\lambda} P(D|\lambda)$

$$P(D|\lambda) = P(x_1, x_2, ..., x_n|\lambda) = \frac{\lambda^{x_1} e^{(-\lambda)}}{x!} \cdot \frac{\lambda^{x_2} e^{(-\lambda)}}{x!} \cdot ... \cdot \frac{\lambda^{x_n} e^{(-\lambda)}}{x!} = \frac{\lambda^{(x_1+x_2+...+x_n)} e^{(-\lambda n)}}{x_1! x_2! ... x_n!}$$

Θέλουμε να μεγιστοποιήσουμε την παραπάνω ποσότητα ως προς λ. Έτσι, μπορούμε να αφαιρέσουμε τον παρονομαστή, καθώς είναι ανεξάρτητος του λ. Μπορούμε, επίσης, να μεγιστοποιήσουμε ισοδύναμα τον ln() της ποσότητας, έχοντας αφαιρέσει τον παρονομαστή.

$\ln(J) = \ln(\lambda^{(x_1+x_2+...+x_n)} \cdot e^{(-\lambda n)} = (x_1 + x_2 + ... + x_n) \ln(\lambda) + \ln(e^{(-\lambda n)} = (x_1 + x_2 + ... + x_n) \ln(\lambda) + (-\lambda n) \cdot 1$
$\frac{dJ_2}{d_\lambda} = 0 \Leftrightarrow \frac{x_1+x_2+...+x_n}{\lambda} - n$

Το μέγιστο θα βρίσκεται στο σημείο, όπου η παράγωγος της J μηδενίζεται.
$\frac{dJ}{d_\lambda} = 0 \Leftrightarrow \frac{x_1+x_2+...+x_n}{\hat{\lambda_{ML}}} - n = 0 \Leftrightarrow \hat{\lambda_{ML}} = \frac{x_1+x_2+...+x_n}{n} \Leftrightarrow$
$\hat{\lambda_{ML}} = \frac{1}{n} \sum_{i=1}^{n} x_i$

B. Ο κανόνας MAP μεγιστοποιεί την a-posteriori πιθανότητα $P(\lambda|D)$, όπου: $P(\lambda|D) = \frac{P(\lambda)P(D|\lambda)}{P(D)}$, ο όρος P(D) είναι ανεξάρτητος των λ, άρα τον αφαιρούμε:

$$P(\lambda|D) = P(\lambda)P(D|\lambda) = \frac{e^{(-\lambda)}.\lambda^{(x_1+x_2+\ldots+x_n)}.e^{(-\lambda n)}}{x_1!x_2!\ldots x_n!} = \frac{\lambda^{(x_1+x_2+\ldots+x_n)}.e^{(-\lambda(n+1))}}{x_1!x_2!\ldots x_n!}$$

Θέλουμε να μεγιστοποιήσουμε την ποσότητα $p(x|\lambda) = \frac{\lambda^x \cdot e^{-\lambda}}{x!}$. Ισοδύναμε μπορούμε να μεγιστοποιήσουμε τον λογάριθμο της ποσότητας. Είναι δυνατό, επίσης, να αφαιρεθεί ο παρονομαστής, διότι είναι ανεξάρτητος του λ (η παράγωγός του ως προς λ είναι μηδέν). Έτσι, προκύπτει ότι:

$$\frac{dJ_2}{d_\lambda} = \frac{d}{d_\lambda}(\ln \lambda^{(x_1+x_2+\ldots+x_n)} e^{-\lambda(n+1)}) =$$
$$\frac{d}{d_\lambda}((x_1 + x_2 + \ldots + x_n)\ln\lambda - \lambda(n+1)) = \frac{x_1+x_2+\ldots+x_n}{\lambda} - (n+1)$$

Για να βρούμε πού μεγιστοποιείται η πιθανότητα, θέτουμε την παράγωγο ίση με μηδέν.

$$\frac{dJ_2}{d_\lambda} = 0 \Leftrightarrow \frac{x_1+x_2+\ldots+x_n}{\lambda} - (n+1) = 0 \Leftrightarrow \lambda = \frac{x_1+x_2+\ldots+x_n}{n+1} \Leftrightarrow$$
$$\lambda_{MAP} = \frac{1}{n+1}\sum_{i=1}^{n} x_i$$

Θέμα 4:Ομαδοποίηση (Clustering) με K-means.

a. Αρχικά, φορτώνεται ένα παράδειγμα συνόλου δεδομένων χρησιμοποιώντας την εντολή load('ex24data1.mat'). Το αρχείο δεδομένων περιέχει ένα σύνολο παραδειγμάτων, τα οποία θα χρησιμοποιηθούν για την εφαρμογή του αλγορίθμου k-means. Στη συνέχεια, ορίζεται ο αριθμός των κέντρων K που θέλουμε να χρησιμοποιήσουμε (στην περίπτωση μας, 3 κέντρα). Επιλέγουμε, επίσης, ένα αρχικό σύνολο initialcentroids για τα κέντρα και χρησιμοποιούμε την συνάρτηση findClosestCentroids.m, για να βρούμε τα κοντινότερα κέντρα για κάθε παράδειγμα στο σύνολο δεδομένων. Έχοντας συμπληρώσει την συνάρτηση findClosestCentroids, αυτή υπολογίζει τον δείκτη idx για κάθε παράδειγμα σε ένα σύνολο δεδομένων, δηλαδή τον δείκτη του κοντινότερου κέντρου για κάθε παράδειγμα. Τα κέντρα που υπολογίστηκαν είναι τα ακόλουθα:

```
Finding closest centroids.

Closest centroids for the first 3 examples:
 1 3 2
(the closest centroids should be 1, 3, 2 respectively)
Program paused. Press enter to continue.
```

b. Μετά την εύρεση των κοντινότερων κέντρων στο προηγούμενο βήμα, χρησιμοποιούμε τη συνάρτηση computeCentroids, για να υπολογίσουμε τα νέα κεντροειδή μέσα. Αυτό γίνεται υπολογίζοντας το μέσο των σημείων δεδομένων που έχουν αντιστοιχηθεί σε κάθε κεντροειδές.

```
Computing centroids means.

Centroids computed after initial finding of closest centroids:
 2.428301 3.157924
 5.813503 2.633656
 7.119387 3.616684

(the centroids should be
   [ 2.428301 3.157924 ]
   [ 5.813503 2.633656 ]
   [ 7.119387 3.616684 ]
```

c. Τέλος, συμπληρώνουμε την εντολή "$X_{(recovered)} = ...$" , η οποία ανακτά την συμπιεσμένη εικόνα από τους δείκτες idx και τα κεντροειδή centroids. Αντιστοιχίζει κάθε εικονοστοιχείο (pixel) (που προσδιορίζεται από τον δείκτη idx) με την τιμή του αντίστοιχου κεντροειδούς. Έτσι, αντί για τα πραγματικά χρώματα των pixel, έχουμε την αναπαράστασή τους ως τιμές κεντροειδών.

Τρέχοντας τον αλγόριθμο για διαφορετικό αριθμό κλάσεων/χρωμάτων K παρατηρείται ότι, όταν αυξάνεται ο αριθμός K, θα δημιουργηθούν περισσότερα κέντρα και οι εικόνες θα συμπιέζονται με περισσότερα χρώματα. Αυτό οδηγεί σε πιο λεπτομερείς αναπαραστάσεις της αρχικής εικόνας, αλλά ταυτόχρονα μπορεί να αυξήσει τον όγκο των δεδομένων που απαιτούνται για τη συμπίεση. Αντίστοιχα, η μείωση του αριθμού των κλάσεων/χρωμάτων K οδηγεί σε μια πιο απλουστευμένη αναπαράσταση της αρχικής εικόνας, αλλά ταυτόχρονα μπορεί να προκαλέσει απώλεια λεπτομερειών, καθώς δημιουργούνται λιγότερα κέντρα και η εικόνα συμπιέζεται με λιγότερες αποχρώσεις.

Θέμα 5: Υλοποίηση ενός απλού νευρωνικού δικτύου.

Μέρος Α.

α. Θέτουμε $z^{(i)} = x^{(i)}W + b$, οπότε $\hat{y} = f(z^{(i)}$. Είναι:

$$J(Y, \hat{Y}; W, b) = \frac{1}{B}\sum_i(-y^{(i)}\ln(y^{(i)}) - (1-y^{(i)})\ln(1-y^{(i)}))$$
$$= \frac{1}{B}\sum_i(-y^{(i)}\ln(\frac{1}{1+e^{-(x^{(i)}W+b)}}) - (1-y^{(i)})\ln(1-\frac{1}{1+e^{-(x^{(i)}W+b)}})), (1)$$

- $ln(\frac{1}{1+e^{(x^{(i)}W+b)}}) = \ln 1 - \ln(1+e^{-(x^{(i)}W+b)}) = -\ln(1+e^{(-z^{(i)})})$

- $\ln(1-\frac{1}{1+e^{-(x^{(i)}W+b)}}) = \ln(\frac{1+e^{(-z^{(i)})}-1}{1+e^{(-z^{(i)})}}) = \ln(e^{(-z^{(i)})})-\ln(1+e^{(-z^{(i)})}) = -z^{(i)} - \ln(1+e^{(-z^{(i)})})$

Άρα, η (1) γίνεται:
$J(Y,\hat{Y};W,b) = \frac{1}{B}\sum_i(-y^{(i)}(-\ln(1+e^{(-z^{(i)})}) - (1-y^{(i)})(-z^{(i)} - \ln(1+e^{(-z^{(i)})}))) =$
$= \frac{1}{B}\sum_i(y^{(i)}(\ln(1+e^{(-z^{(i)})}) + (y^{(i)}-1)(-z^{(i)} - \ln(1+e^{(-z^{(i)})})) =$
$= \frac{1}{B}\sum_i(y^{(i)}(\ln(1+e^{(-z^{(i)})}) - y^{(i)}z^{(i)} + z^{(i)} - y^{(i)}\ln(1+e^{(-z^{(i)})})) + ln(1+e^{(-z^{(i)})}))$
$\Leftrightarrow J(Y,\hat{Y};W,b) = \frac{1}{B}\sum_i(z^{(i)} - z^{(i)}y^{(i)} + \ln(1+e^{(z^{(i)})}))$

β. $\frac{\partial J}{\partial z^{(i)}} = \frac{\partial}{\partial z^{(i)}}(\frac{1}{B}\sum_i(z^{(i)} - z^{(i)}y^{(i)} + \ln(1+e^{(z^{(i)})}))$

Λόγω της γραμμικότητας των τελεστών έχουμε ότι:
$\frac{\partial J}{\partial z^{(i)}} = \frac{1}{B}\sum_i \frac{\partial}{\partial z^{(i)}}(z^{(i)} - z^{(i)}y^{(i)} + \ln(1+e^{-z^{(i)}})) =$
$= \frac{1}{B}\sum_i \frac{\partial}{\partial z^{(i)}}z^{(i)} - \frac{1}{B}\sum_i \frac{\partial}{\partial z^{(i)}}(z^{(i)}y^{(i)}) + \frac{1}{B}\sum_i \frac{\partial}{\partial z^{(i)}}\ln(1+e^{-z^{(i)}} =$
$= \frac{1}{B}\sum_i 1 - \frac{1}{B}\sum_i y^{(i)} + \frac{1}{B}\sum_i \frac{1}{1+e^{-z^{(i)}}}(-e^{-z^{(i)}}) =$
$= 1 - y^{(i)} - \frac{1}{B}\sum_i(1 - \frac{1}{1+e^{-z^{(i)}}}) = 1 - y^{(i)} - \frac{1}{B}\sum_i 1 + \frac{1}{B}\sum_i y^{(i)} = 1 - y^{(i)} - 1 + \hat{y^{(i)}}$
Οπότε: $\frac{\partial J}{\partial z^{(i)}} = -y^{(i)} + \hat{y^{(i)}}, \quad i\epsilon\text{Batch}.$

γ. Ο κανόνας παραγώγισης με τον πίνακα W είναι: $\frac{\partial(xW)}{\partial W} = x^T$
Με τον κανόνα της αλυσίδας υπολογίζουμε τις μερικές παραγώγους $\frac{\partial J}{\partial W}$ και $\frac{\partial J}{\partial b}$.

- $\frac{\partial(J)}{\partial(W)} = \frac{\partial J}{\partial z}\cdot\frac{\partial z}{\partial W}$
  Άρα: $\frac{\partial J}{\partial W} = (-y^{(i)} + \hat{y^{(i)}})\cdot\frac{\partial}{\partial W}(xW+b) = (-y^{(i)} + \hat{y^{(i)}})\cdot X^T =$
  $= (-y^{(i)} + \hat{y^{(i)}})\cdot X^T = -y^{(i)}X^T + \hat{y^{(i)}}X^T$

- $\frac{\partial J}{\partial b} = \frac{\partial J}{\partial z}\cdot\frac{\partial z}{\partial b} = (-y^{(i)} + \hat{y^{(i)}})\cdot\frac{\partial}{\partial W}(xW+b) = (-y^{(i)} + \hat{y^{(i)}})\cdot 1 =$
  $= -y^{(i)} + \hat{y^{(i)}}$

δ. Ο αλγόριθμος του Back Propagation αποτελείται από τα εξής βήματα:

1. Αρχικοποιούμε, τυχαία, τα βάρη κάθε νευρώνα του Νευρωνικού δικτύου (NN).
2. Περνάμε τις εισόδους από το NN και παίρνουμε το output.
3. Υπολογίζουμε το σφάλμα σύμωνα με τον τύπο της cross-entropy ανάμεσα στην πραγματική και στην θεωρητική έξοδο.
4. Ενημερώνουμε τα βάρη σύμφωνα με ένα ποσοστό του gradient του σφάλματος.
5. Επαναλαμβάνουμε τα $2-4$ μέχρι να ελαχιστοποιηθεί το σφάλμα.

Στο layer που βρίσκεται πιο κοντά στην έξοδο (εδώ το Layer 3) τα βάρη και το bias υπολογίζονται σύμφωνα με τον εξής τύπο:

$$L_3^{(n+1)} = L_3^{(n)} - C_n \cdot \nabla_{L_3} ||y - \hat{y}||^2, \text{ όπου } L_3 = \begin{bmatrix} W_3 \\ \vec{b_3^T} \end{bmatrix}$$

Μέρος Β.

Συμπληρώσαμε τον κώδικα και, τρέχοντάς τον, παρατηρήσαμε ότι η μέση ακρίβεια πρόβλεψης των test δεδομένων είναι 72% (πήραμε την αριθμιτική μέση τιμή του ratio από 5 runs).

Έπειτα δοκιμάσαμε να αλλάξουμε διάφορες παραμέτρους του νευρωνικού δικτύου:

1.
   - Αλλάζοντας το epoch από 100 σε 1000 με σταθερές τις τιμές του batch_size και learning_rate παρατηρούμε ότι η ακρίβεια αυξανεται στο 95%.
   - Διατηρούμε το epoch στο 1000 και αλλάζουμε το learning_rate σε 0.01. Βλέπουμε ότι η ακρίβεια μειώνεται στο 68%.
   - Έπειτα αλλάζουμε και την τιμή του batch_size (με epoch=100, learning_rate=0.1). Αρχικά την μειώνουμε σε 32 και μετράμε την ακρίβεια πρόβλεψης στο 68%.
     Τέλος αυξάνοντάς το σε 512 βλέπουμε ότι η ακρίβεια αυξάνεται σε 78% (Αν το αυξήσουμε σε 1000 μετράμε πολύ αρνητικά αποτελέσματα)

   Παρατηρούμε ότι, όσο αυξάνεται το γινόμενο epoch· learning_rate ,τόσο αυξάνεται και η ακρίβεια.

   Επίσης, η αύξηση του batch_size έχει ως αποτέλεσμα την ταχύτερη εκπαίδευση του δικτύου, καθώς και σχετικά καλύτερη ακρίβεια. Αυτό οφείλεται στο γεγονός ότι, για μεγάλο batch_size περνάνε περισσότερα δείγματα κάθε φορά από τους νευρώνες και υπολογίζεται ακριβέστερα το gradient του σφάλματος.

2. Χρησιμοποιούμε συνάρτηση ενεργοποίησης tanh και συνάρτηση σφάλματος MSE (Mean Square Error).

   Παρατηρούμε ότι για τις αρχικές τιμές η ακρίβεια είναι χειρότερη (54%).

   Αυξάνοντας το epoch σε 1000 η ακρίβεια αυξάνεται (63%).

   Παρατηρούμε ότι, με συνάρτηση ενεργοποίησης tanh και συνάρτηση σφάλματος MSE παίρνουμε χειρότερα αποτελέσματα.

3. Θα δοκιμάσουμε διαφορετικές αρχιτεκτονικές δικτύου (με συνάρτηση ενεργοποίησης την σιγμοειδή και συνάρτηση σφάλματος την cross entropy).

   - Δοκιμάζουμε να αφαιρέσουμε το 2ο dense layer. Η ακρίβεια μειώνεται (54%) και το συνολικό σφάλμα αυξάνεται.
   - Προσθέτουμε και τρίτο dense layer στο δίκτυο. Παρατηρούμε ότι, όπως είναι λογικό, η ακρίβεια αυξάνεται αρκετά (82%).

Θέμα 6: Convolutional Neural Networks for Image Recognition.

1. Αρχικά, τρέξαμε τον κώδικα του αρχείου fashion.py για epoch=400 και αλγόριθμο βελτιστοποίησης adam και σημειώσαμε την τιμή το σφάλματος και της ακρίβειας (Οι τιμές από κάθε υποερώτημα φαίνονται στο πινακάκι στο τέλος). Λάβαμε τα εξής plots:

Figure 8: Εικόνες ρούχων.

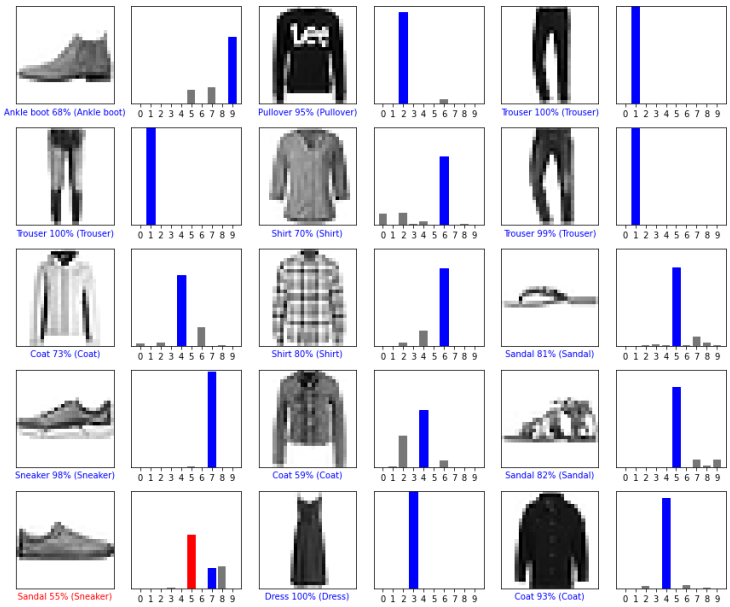
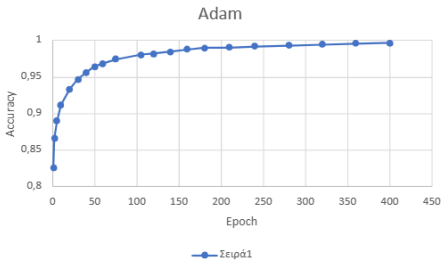


Figure 9: Προβλέψεις εικόνων ρούχων.

Παρατηρούμε ότι, η ακρίβεια στα δεδομένα εκπαίδευσης μεγαλώνει όλο και πιο αργά, όσο πλησιάζει την τιμή 1, ενώ η ακρίβεια στα δεδομένα επικύρωσης ανεβαίνει αρχικά και μετά παραμένει αρχικά σταθερή.



(a) Accuracy/epoch.

(b) Validation Accuracy/epoch.

2. Έπειτα, δοκιμάσαμε τους αλγορίθμους βελτιστοποίησης adam, sgd, rmsprop, nadam, adamax, και ftrl. Όπως φαίνεται στο τελευταίο πινακάκι, καλύτερο accuracy και validation accuracy δίνει ο adamax.

3. Στη συνέχεια αλλάξαμε την αρχιτεκτονική του δικτύου, προσθέτοντας κώδικα στο αρχείο fashion_cnn.py. Συγκεκριμένα υλοποιούμε ένα Convolutional Neural Network με adam optimizer, και θέτουμε το epoch σε 50 (από 400) για ταχύτερη εκτέλεση.

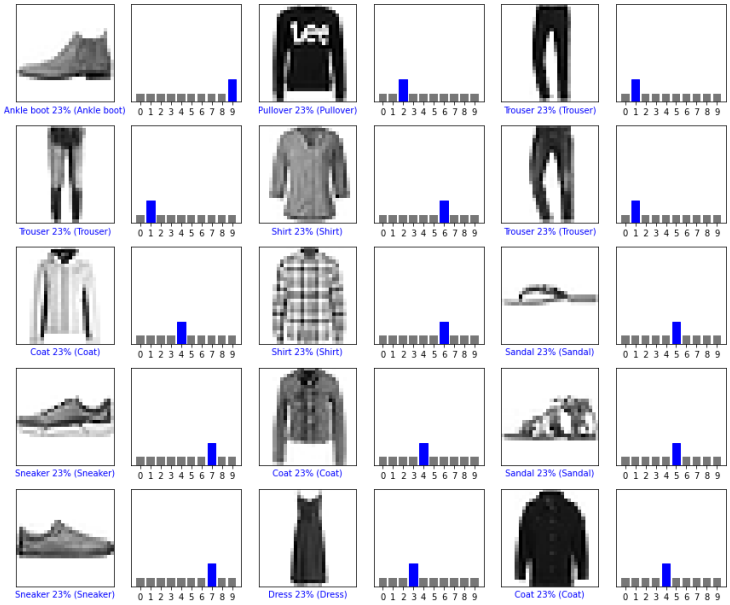


Figure 11: Προβλέψεις εικόνων ρούχων.

Το δίκτυο αυτό απαιτεί περισσότερο χρόνο για την εκπαίδευσή του, όμως δίνει καλύτερο test accuracy = 92.4% σε μόλις 50 epochs.

4. Έπειτα, προσθέτουμε Batch Normalization αμέσως μετά τα επίπεδα conv1, conv2, conv3, conv4, conv5 και dense1.

5. Τέλος, προσθέτουμε dropout(0.2) αμέσως μετά το πρώτο maxPooling, dropout(0.3) αμέσως μετά το δεύτερο maxPooling, dropout(0.4) αμέσως μετά το τρίτο maxPooling και dropout(0.5) αμέσως πριν το τελευταίο dense layer.

   Οι μετρήσεις από όλα τα ερωτήματα καταγράφονται παρακάτω:

Ερώτημα	Optimizer	Max Epoch	Train Accuracy	Test Accuracy
1, 2	adam	400	0.9963	0.8833
2	sgd	400	0.9934	0.8829
2	rmsprop	400	0.9936	0.8803
2	nadam	400	0.9959	0.8817
2	adamax	400	0.9999	0.8842
2	ftrl	400	0.8587	0.8412
3	adam	50	0.9933	0.9241
4	adam	50	0.9962	0.9346
5	adam	50	0.9959	0.9246